



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-19

今日榜单

- 1** **cloudflare/security-audit-skill** JavaScript 4天
A coding-agent skill for multi-phase security audits with independently verified...
- 2** **trycua/cua** HTML NEW
Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks...
- 3** **addyosmani/agent-skills** JavaScript 3天
Production-grade engineering skills for AI coding agents.
- 4** **coder/coder** Go NEW
Secure environments for developers and their agents
- 5** **anthropics/claude-code** TypeScript 3天
Claude Code is an agentic coding tool that lives in your terminal, understands y...
- 6** **Open-Dev-Society/OpenStock** TypeScript NEW
OpenStock is an open-source alternative to expensive market platforms. Track rea...
- 7** **higgsfield-ai/higgsfield** Jupyter Notebook NEW
Fault-tolerant, highly scalable GPU orchestration, and a machine learning frame...
- 8** **docling-project/docling** Python NEW
Get your documents ready for gen AI
- 9** **cloudflare/quiche** Rust NEW
🍷 Savoury implementation of the QUIC transport protocol and HTTP/3
- 10** **asciimoo/hister** Go NEW
Your own search engine

#1

JavaScript

连续 4 天

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **15,234** Forks **834** Today **+3,162**

<https://github.com/cloudflare/security-audit-skill>

项目简介：这是 Cloudflare 开源的一个「编码智能体技能包」（coding-agent skill），它把通用的 AI 编码助手转变成为一名结构化安全审计员。它解决的核心问题是：让 AI 审计代码时不再靠"感觉"下结论，而是通过多阶段、可验证、机器可读的流程产出可信的漏洞发现。项目源自 Cloudflare 内部漏洞挖掘系统（vulnerability harness）的雏形，是那套大规模系统的单仓库起点。

核心功能：

- **六阶段审计流水线：**从侦察（Reconnaissance）→ 覆盖驱动的漏洞狩猎（Hunting）→ 候选验证（Validation）→ 结构化输出 → 独立记录核验 → 目标中立报告，环环相扣。
- **独立验证机制：**每个候选漏洞都交给全新的"验证者"智能体去尝试证伪（disprove），最终结论还要经过独立核验，避免自我确认偏差。
- **机器可读的发现记录：**输出 findings.json，并用 report-schema.json 严格校验，区分 confirmed（已确认）、needs_validation（待验证）、rejected（已否决）三种判定。
- **覆盖台账（coverage ledger）：**用 coverage-ledger.json 追踪哪些代码区域已检查、哪些是盲区，并支持多次运行叠加（additive），增量针对未覆盖区域。
- **庞大的攻击类别知识库：**涵盖内存安全、AI/LLM 提示注入、Web 协议认证、客户端、供应链、云部署、资源耗尽、数据隔离等十余个专门文档。

适用场景：主要面向安全工程师、渗透测试人员和安全意识强的开发团队。使用场景包括：对自有代码库做系统性安全审计、在 CI 或发布前发现漏洞、对 LLM 应用（如 agent/工具调用）做提示注入检查，以及需要可追溯、可复现审计证据的合规性场景。

技术亮点：最值得关注的是它的"证伪优先"设计哲学——不靠单个智能体自说自话，而是用隔离的、角色分工的多个智能体互相制衡（hunter 与 critic、candidate 与 verifier）。其次是**零依赖的校验脚本**（validate-coverage-ledger.cjs、validate-findings.cjs），用纯 Node.js 保证输出格式的严格一致性。此外，它把审计知识按攻击面拆分成独立文档，使技能可扩展、可组合。

上手建议：最快的方式是通过 Skills CLI 安装——`npx skills add https://github.com/cloudflare/security-audit-skill --skill security-audit`，然后在目标代码库中让编

码智能体执行「security audit this codebase」。想深入理解或贡献，建议先读 SKILL.md 掌握核心原则和工作流，再对照 RECONNAISSANCE.md、HUNTING.md、VALIDATION-AND-REPORTING.md 理解各阶段逻辑；若要扩展攻击面，可参考 ATTACK-CLASSES.md 的写法新增类别文档。配合 Cloudflare 官方博客《Build your own vulnerability harness》一起阅读，能更好理解其设计意图。

#2

HTML

NEW

cua

Scale computer-use 2.0 with open-source drivers, cross-OS fleets, and benchmarks for training, evaluation, and data generation.

Stars **23,934** Forks **1,655** Today **+383**

<https://github.com/trycua/cua>

项目简介：Cua (trycua/cua) 是一个为 AI Agent 提供"可操作计算机"能力的开源基础设施项目。它解决的核心问题是：让 AI 智能体能够像人一样操作真实的桌面环境——点击、输入、截图、运行命令，而不仅仅是调用 API 或生成文本。项目定位在"Computer-Use 2.0"，即让 Agent 在同一任务中自由穿梭于代码、API 和图形界面之间。

核心功能： - **Cua Fleets**：在云端批量配置隔离的 Linux 桌面沙箱，代码可从资源池中"认领"桌面并通过 Sandbox SDK 执行命令、截图和操作应用。 - **Cua Driver**：跨 macOS、Windows、Linux 的桌面自动化驱动，用于检查和操控应用程序（如操作 LibreOffice、Inkscape）。 - **Lume**：在 Apple Silicon 上运行本地 macOS/Linux 虚拟机，支持 SSH 连接，适合本地开发和测试。 - **Cua Bench**：面向 computer-use agent 的基准测试框架，支持创建任务、评估 Agent 表现并导出轨迹数据，用于训练和评测。

适用场景：主要面向 AI Agent 研究者、computer-use 模型开发者，以及需要大规模数据生成和评估的团队。典型场景包括：训练 GUI 操作类 Agent、在隔离环境中安全测试自动化脚本、批量生成操作轨迹数据、以及为多平台 Agent 提供统一的桌面操作接口。

技术亮点：一是"Fleets"的池化沙箱设计，支持按需认领和释放云桌面，兼顾隔离性与规模化；二是本地沙箱与云端 Fleet 共享同一套 Sandbox SDK，降低了从开发到部署的迁移成本；三是跨 OS 驱动（macOS/Windows/Linux）加上 Apple Silicon 上的本地 VM 支持，覆盖面较广；四是内置 benchmark 与轨迹导出，把"评测"和"数据生成"作为一等公民，而非事后补充。

上手建议：建议从官方文档的四个入门教程切入——先跑通>Your first Cloud Fleet"（云端桌面 + 命令 + 截图），再尝试 Cua Driver 操作一个简单应用，最后用 Cua Bench 创建并验证一个模拟任务。注意 Fleet 的池在 claim 结束后可能仍保留付费容量，务必按教程执行清理步骤。贡献方面可关注 Sandbox SDK 的 runtime 支持扩展、新的 benchmark 任务集，以及跨平台驱动的兼容性改进。

#3

JavaScript

连续 3 天

agent-skills

Production-grade engineering skills for AI coding agents.

Stars **96,724** Forks **10,219** Today **+547**

<https://github.com/addyosmani/agent-skills>

项目简介：这是 Google Chrome 团队知名工程师 Addy Osmani 推出的开源项目，把资深工程师在软件开发全流程中用到的工程实践、质量门禁和最佳实践，封装成 AI 编程 Agent 可直接调用的"技能包"。它解决的核心问题是：AI 写代码虽然快，但往往缺乏系统性的工程纪律，这个项目让 AI Agent 在开发的每个阶段都能遵循一致的高标准流程。

核心功能： 1. **9 个斜杠命令覆盖完整开发生命周期：** `/spec`（定义需求）、`/plan`（拆解任务）、`/build`（增量构建）、`/test`（测试验证）、`/review`（代码审查）、`/ship`（发布上线）等，每个命令自动激活对应的技能集。 2. **25 个可独立安装的工程技能：**如 `code-review-and-quality`（多维代码审查）、`test-driven-development`（强制红绿重构）、`interview-me`（需求追问）等，既可整体安装也可按需选取。 3. **自动技能激活机制：**根据当前工作内容自动触发相关技能，比如设计 API 时自动启用接口设计技能，构建 UI 时启用前端工程技能。 4. **`/build auto` 自主执行模式：**批准计划后可自动完成所有任务的实现，但每个任务仍保持测试驱动和独立提交，遇到失败或高风险步骤会暂停等待人工介入。

适用场景：主要面向使用 Claude Code、Cursor、Codex、Copilot、Cline 等 AI 编程工具的开发者，尤其是希望让 AI 助手产出"生产级"而非"演示级"代码的团队。适合从需求定义到上线发布的完整项目开发，也适合在代码审查、性能审计、代码简化等特定环节单独使用。

技术亮点：项目采用"技能即配置"的设计哲学，通过标准化的技能描述让不同 AI Agent 平台都能理解和执行（借助 `vercel-labs/skills CLI` 可安装到 70+ 种 Agent 中）。其流程编排思路——将软件工程最佳实践编码为 Agent 可执行的命令序列——是当前 AI 辅助开发领域的一个重要范式探索。`/build auto` 在自动化与安全验证之间找到了平衡点：去掉人工在任务间的切换，但保留测试验证和风险暂停机制。

上手建议：最快的方式是运行 `npx skills add addyosmani/agent-skills` 一键安装全部技能，或先用 `--list` 浏览再按需安装单个技能。Claude Code 用户可通过插件市场直接安装获得最佳体验。想贡献的话，可以从新增技能或改进现有技能的工作流定义入手，注意单个技能安装时 `references/` 目录的共享清单路径问题（见 issue #361）。

#4

Go

NEW

coder

Secure environments for developers and their agents

Stars **15,461** Forks **1,509** Today **+406**

<https://github.com/coder/coder>

项目简介：Coder 是一个可自托管的云开发环境平台，目标是让开发者用 Terraform 定义开发环境，并在云端快速创建、连接和销毁这些环境。它同时把 AI 编码代理（Coder Agents）纳入同一套基础设施中，解决远程开发环境配置复杂、资源闲置浪费以及 AI 工具凭据管理混乱的问题。

核心功能： - 用 Terraform 定义工作区，可运行在 EC2、Kubernetes、Docker 等多种基础设施上。 - 通过 WireGuard 隧道安全连接工作区，并在闲置时自动关闭以节省成本。 - 内置 Coder Agents，支持接入 Anthropic、OpenAI、Google、Bedrock 或自托管模型，在控制平面执行 AI 编码循环。 - 提供 AI Gateway，集中管理模型认证、审计日志和成本追踪，避免在工作区中存放 LLM 凭据。

适用场景：适合需要为团队统一管理开发环境的中大型工程组织，尤其是希望自托管、重视安全合规和成本控制的团队。也适合想在生产级基础设施上安全运行 AI 编码代理，而不愿把模型密钥散落在各个工作区里的开发者。

技术亮点：项目用 Go 编写，核心思路是把开发环境抽象为 Terraform 模板，实现基础设施即代码；WireGuard 隧道保证连接安全，自动关停机制控制云成本。AI 代理部分把执行循环放在控制平面，结合用户身份和审计日志，在架构上把“AI 工具治理”和“开发环境治理”统一了起来。

上手建议：最快的方式是在 Linux 或 macOS 上运行 `curl -L https://coder.com/install.sh | sh`，然后执行 `coder server` 并访问 `http://localhost:3000` 创建初始用户和 Docker 模板。生产部署建议配置 PostgreSQL 13+ 和外部访问 URL，并参考官方“validated architectures”文档。想贡献代码可从 GitHub Issues 和 Discord 社区入手，先熟悉模板系统和工作区管理流程。

#5

TypeScript

连续 3 天

claude-code

Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands.

Stars **146,545** Forks **23,851** Today **+482**

<https://github.com/anthropics/claude-code>

项目简介：Claude Code 是 Anthropic 官方推出的终端智能体编程工具，将 Claude 大模型的能力直接嵌入开发者的命令行工作流中。它解决的核心问题是让开发者用自然语言完成代码理解、任务执行和 Git 操作，而不必在 IDE 与浏览器之间来回切换。

核心功能：一是自然语言驱动的代码任务执行，可自动完成常规编码工作；二是对代码库的深度理解与复杂代码解释；三是内置 Git 工作流处理（提交、分支、PR 等）；四是多入口接入，支持终端、IDE 以及 GitHub 上 @claude 触发；五是插件机制，可通过自定义命令和 agent 扩展能力。

适用场景：主要面向日常需要频繁处理代码库、提交代码、排查问题的开发者，尤其是习惯终端工作流的工程师。典型场景包括快速理解陌生代码库、批量执行重复性重构、生成提交信息与处理 PR，以及在 CI/协作流程中通过 GitHub 触发自动化任务。

技术亮点：项目以 TypeScript 编写，采用 agentic（智能体）架构而非简单的命令补全，意味着它能自主规划并执行多步操作。安装方式已从 npm 迁移到官方脚本、Homebrew 和 WinGet，反映出它正从“Node 包”向独立 CLI 工具演进；同时 README 明确披露了数据采集与隐私保护策略，体现了企业级合规考量。

上手建议：最简路径是按官方推荐方式安装（macOS/Linux 用 curl 脚本或 Homebrew，Windows 用 PowerShell 或 WinGet），进入项目目录运行 `claude` 即可。想深入可查阅官方文档与 `plugins` 目录了解插件扩展；遇到问题用 `/bug` 命令反馈或提 GitHub issue，也可加入 Discord 社区交流。

#6

TypeScript

NEW

OpenStock

OpenStock is an open-source alternative to expensive market platforms. Track real-time prices, set personalized alerts, and explore detailed company insights — built openly, for everyone, forever free.

Stars **15,751** Forks **2,061** Today **+477**

<https://github.com/Open-Dev-Society/OpenStock>

项目简介：OpenStock 是一个开源的美股/金融市场追踪平台，定位为昂贵商业行情软件（如 Bloomberg、各类付费终端）的免费替代品。它让普通用户无需订阅费就能查看实时股价、设置提醒并了解公司基本面，目前以 AGPL-3.0 协议开放，社区驱动、永久免费。

核心功能：实时价格追踪与自选股看板；个性化价格提醒（通过邮件推送）；公司详情与基本面洞察；基于 TradingView 的交互式图表；用户认证与个人账户体系。

适用场景：适合个人投资者、量化爱好者、学生等希望低成本监控持仓和行情的用户；也适合开发者作为学习 Next.js 全栈 + 金融数据 API 集成的参考项目，或在此基础上二次开发自建行情工具。

技术亮点：采用 Next.js + TypeScript + Tailwind + shadcn/ui 的现代前端栈，图表用 TradingView 组件、行情数据来自 Finnhub；后端用 MongoDB 存储、Better Auth 做认证、Inngest 处理后台任务与定时提醒、Nodemailer 发邮件，整体是典型的 Serverless 友好型全栈架构。AGPL-3.0 协议对商业二次部署有传染性约束，值得注意。

上手建议：先克隆仓库，配置 Finnhub API Key、MongoDB 连接串和邮件服务等环境变量，本地跑起来体验核心流程；想贡献可从 README 的 issue 列表、good-first-issue 标签或文档/UI 改进入手，注意提交前遵循其 AGPL 授权与署名要求。

#7

Jupyter Notebook

NEW

higgsfield

Fault-tolerant, highly scalable GPU orchestration, and a machine learning framework designed for training models with billions to trillions of parameters

Stars **4,725** Forks **876** Today **+325**

<https://github.com/higgsfield-ai/higgsfield>

项目简介：Higgsfield 是一个面向大规模模型训练（数十亿到万亿参数）的开源 GPU 编排与机器学习框架，核心目标是让多节点分布式训练变得“不用哭着调”。它同时承担 GPU 资源管理和训练框架两个角色，试图把环境配置、任务调度和分布式训练统一到一个工作流里。

核心功能：一是 GPU 资源分配与队列管理，支持独占/非独占节点访问并处理资源竞争；二是对 ZeRO-3 DeepSpeed 和 PyTorch FSDP 的原生支持，方便万亿参数级模型分片；三是通过 GitHub/GitHub Actions 实现实验的自动部署、运行与监控，并保存检查点；四是提供简洁的实验定义接口，屏蔽 DeepSpeed、accelerate 等底层细节。

适用场景：适合拥有多台 GPU 节点（Ubuntu + SSH + 非 root sudo 用户）的团队，尤其是需要训练 LLM 的研究者或工程团队。典型场景包括在 Azure、LambdaLabs、FluidStack 等云上做 LLaMa 类模型的分布式微调，以及希望通过 GitHub 做 ML 持续集成和实验追踪的开发者。

技术亮点：它把“环境地狱”和“配置地狱”作为主要痛点来解决，用 @experiment 装饰器和预置的 Llama70b、LlamaLoader 等类大幅简化训练代码；同时把节点安装、部署密钥、运行工作流都自动化到 GitHub 流程中，让实验的版本、依赖和检查点可追踪、可复现，而不是依赖 Hydra 式的复杂 YAML 配置。

上手建议：从 README 中的 setup.md 和 tutorial.md 开始，先按“初始化项目 → 配置环境 → 配置 Git → 设置节点 → 运行第一个实验”的顺序走一遍。如果只是想快速体验，可以直接 `pip install higgsfield==0.0.3` 并参考 LLaMa 训练示例；想贡献代码的话，建议先熟悉其 GitHub Actions 部署链路和实验抽象层的设计。

#8

Python

NEW

docling

Get your documents ready for gen AI

Stars **66,776** Forks **4,817** Today **+94**

<https://github.com/docling-project/docling>

项目简介：Docling 是 IBM 开源（现属 LF AI & Data 基金会）的文档解析工具，目标是把各种格式的文档“清洗”成适合生成式 AI 使用的结构化数据。它解决的核心痛点是：LLM 无法直接读懂 PDF、扫描件、PPT 等复杂文档，而 Docling 负责在中间做高质量的格式转换与语义提取。

核心功能：一是多格式解析，覆盖 PDF、DOCX、PPTX、XLSX、HTML、EPUB、LaTeX、音频甚至视频；二是高级 PDF 理解能力，包括版面分析、阅读顺序还原、表格结构识别、公式与代码提取；三是统一的 DoclingDocument 中间表示，并支持导出为 Markdown、HTML、JSON、DocTags 等多种格式；四是内置 OCR 与视觉语言模型（如 GraniteDocling）支持，能处理扫描件和图像。

适用场景：主要面向构建 RAG（检索增强生成）或文档问答系统的开发者，需要把企业内部的合同、论文、财报、专利等非结构化文档喂给 LLM。也适合做数据预处理的 ML 工程师、需要本地化处理敏感文档（支持离线/气隙环境）的团队，以及想用 MCP/API 把文档理解接入 Agent 工作流的场景。

技术亮点：用 Pydantic v2 定义了强类型的文档表示模型，保证解析结果可校验、可序列化；采用模块化 pipeline 设计，PDF 后端可插拔（如 layout model、tableformer、OCR 引擎），兼顾精度与性能；同时提供 MCP server 和 docling-serve API，能无缝接入 LangChain、LlamaIndex 等主流框架，工程化程度相当高。

上手建议：最快的方式是 `pip install docling`，然后跑一个把 PDF 转成 Markdown 的最小示例，感受输出质量。深入使用可查阅官方文档的 pipeline 配置和自定义导出部分；想贡献的话，建议先从支持新格式或改进某个 PDF 后端入手，项目有完善的测试和 arXiv 论文（2408.09869）作为背景参考，Discord 社区也很活跃。

#9

Rust

NEW

quiche



Savoury implementation of the QUIC transport protocol and HTTP/3

Stars **11,883** Forks **1,099** Today **+5**

<https://github.com/cloudflare/quiche>

项目简介：quiche 是 Cloudflare 开源的 QUIC 传输协议与 HTTP/3 的 Rust 实现，遵循 IETF 标准。它解决的是在 UDP 之上构建可靠、低延迟、加密传输层的问题，为新一代 Web 协议提供底层支撑。

核心功能：提供处理 QUIC 数据包和管理连接状态的底层 API；支持 HTTP/3 协议栈；内置 TLS 集成（基于 BoringSSL）；支持流量控制、拥塞控制、多路复用流等 QUIC 核心特性；附带命令行客户端/服务端工具用于测试。

适用场景：适合需要自建 HTTP/3 服务端或客户端的开发者，如 CDN 厂商、浏览器/网络库作者、DNS over HTTP/3 实现者。Cloudflare 边缘网络、Android DNS 解析器、curl 的 HTTP/3 支持均基于它。

技术亮点：采用 Rust 编写，内存安全性强；设计为低层库，I/O 和事件循环交由应用层处理，灵活度高；与 Cloudflare 生产环境深度绑定，经过大规模流量验证；配置项默认保守（如流数量为 0），强制开发者显式设定，避免误用。

上手建议：先通过 `cargo run --bin quiche-client` 和 `quiche-server` 体验命令行工具；再阅读 `Config`、`connect()`、`recv()` 等核心 API 文档；贡献可从修复 issue、补充测试或改进文档入手，注意项目使用 BSD-2-Clause 许可证。

#10

Go

NEW

hister

Your own search engine

Stars **5,110** Forks **212** Today **+430**

<https://github.com/asciimoo/hister>

项目简介：Hister 是一个可自托管的个人搜索引擎，专门索引你访问过的网页和你本地保存的文件，把它们的完整内容都纳入检索范围。它解决的是"信息明明见过却再也找不到"的痛点——不依赖 Google 或第三方搜索服务，而是为你自己的数字足迹建立私有索引。

核心功能：一是全文索引，搜索的是网页正文和文件内容，而不只是标题和 URL；二是浏览器扩展自动收录，用 Firefox/Chrome 插件把新访问的页面实时存入索引；三是强大的查询语法，支持字段过滤、短语、通配符、否定、别名和结果优先级；四是可选的语义搜索，通过自配置的 embeddings 接口按"含义"而非关键词找文档；五是提供 Web、终端 TUI、命令行和 MCP 多种客户端入口。

适用场景：适合注重隐私、不希望浏览记录被第三方掌握的用户，尤其是需要频繁回溯已读资料的开发者、研究人员和写作者。典型用法包括：导入浏览器历史快速建立索引、索引本地文档目录、用 MCP 让 AI 助手直接检索你的个人知识库，或在共享服务器上为多用户分别隔离文档和搜索结果。

技术亮点：用 Go 编写，单文件二进制即可运行，部署门槛极低；默认无遥测、无云同步，隐私边界清晰。架构上同时支持传统全文检索与可选语义检索，并通过 MCP 协议把个人索引接入 AI 助手，这是它区别于传统书签/笔记工具的关键设计。多用户隔离和 AGPLv3 授权也体现了自托管社区的取向。

上手建议：最快的方式是从最新 release 下载对应平台二进制，重命名为 hister 后执行 `./hister listen`，打开 `http://127.0.0.1:4433` 并安装浏览器扩展即可，本地个人使用无需任何配置。想深入可查阅官方 quickstart 配置索引范围，或用 Homebrew、Docker、Nix 安装。若想贡献代码，需准备 Go 1.26、npm 和 C 编译器，先读 CONTRIBUTING.md，再通过 IRC 或 Discord 与社区沟通。

数据来源: github.com/trending

报告日期: 2026-09-19

由 DeepSeek AI 提供智能分析 | 自动生成